

Summer term 2026

Visual Data Analysis

Assignment Sheet 7

Solution has to be submitted via eCampus by June 5, 2026, 10:00 a.m.

If you have questions concerning the exercises, please use the forum on eCampus.

- Please work on this exercise in **small groups** of 3 students. Submit each solution only once, but clearly indicate who contributed to it by forming a team in eCampus. Remember that all team members have to be able to explain all answers.
- Please submit your answers in PDF format, and your scripts as *.py/*.ipynb files. If you are using [Jupyter notebook](#), please also export your scripts and results as PDF.

Exercise 1 (Lambert Equal-Area Projection, 5 Points)

In the lecture, we presented equations for several map projections without formally proving their claimed properties. In this exercise, you will verify that the Lambert cylindrical equal-area projection

$$x = \lambda, \quad y = \sin \phi$$

preserves area.

- a) Write down a parametrization

$$\mathbf{p}(\phi, \lambda)$$

of a sphere of radius R using geographical latitude ϕ and longitude λ . (1P)

- b) Compute the infinitesimal surface area element corresponding to infinitesimal changes $d\phi$ and $d\lambda$. One approach is to compute the tangent vectors

$$\frac{\partial \mathbf{p}}{\partial \phi}, \quad \frac{\partial \mathbf{p}}{\partial \lambda},$$

and then compute the norm of their cross product. (2P)

- c) Compute the infinitesimal area element in the projected (x, y) coordinates corresponding to infinitesimal changes $d\phi$ and $d\lambda$, using

$$x = \lambda, \quad y = \sin \phi.$$

Compare your result with the spherical surface area element obtained above. (2P)

Exercise 2 (Mapping Wind Power Generation, 20 Points)

This task introduces you to the **geopandas** package, which makes it quite easy to generate certain types of geospatial data visualizations. The [Getting Started](#) tutorial might be helpful.

For this exercise, we provide geospatial data of all Landkreise (administrative regions) in Germany in [shapefile format](#). In particular, you will find five files with the following extensions:

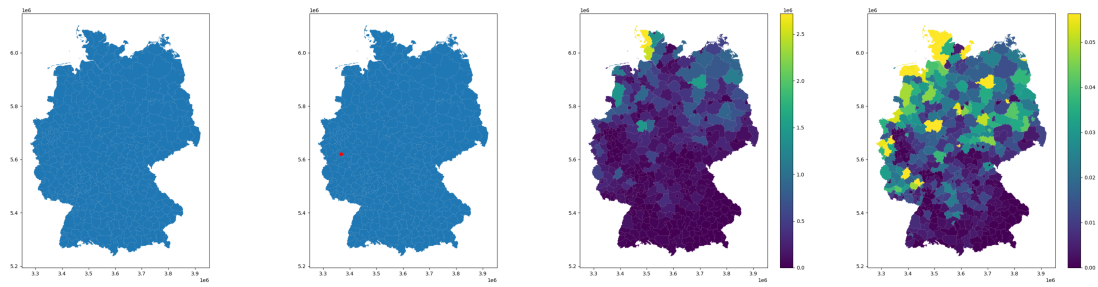


Figure 1: A screenshot of the maps you are asked to create in this task. From left to right: administrative areas of Germany, Bonn marked, net wind power generation by district, normalized by area.

- **.shp**: contains the feature geometry itself
- **.shx**: contains a positional index of the feature geometry to allow seeking forwards and backwards quickly
- **.dbf**: contains attributes for each shape, in dBase IV format
- **.cpg**: specifies the character encoding that is used within the **.dbf** file
- **.prj**: projection description, using a well-known text representation of coordinate reference systems

If you read in the **.shp** file with **geopandas**, the remaining files are read automatically and will yield the polygons as well as some additional information on the Landkreise. Use this data to solve the following tasks:

- Create a plot that shows the administrative areas of Germany. (2P)
 - Find Bonn in the dataset and plot the polygon for Bonn. (2P)
 - Add a marker at the location of Bonn to your administrative map of Germany. (2P)
- Hint:* You somehow need to turn the polygon into a single point.

You will now create your own power generation map: it should use colors to show how much power is generated within each administrative district. To do this, you need to combine the shapefile dataset with the power generation dataset **power_generation.csv**, which is also in the **districts** folder (Source: <https://www.marktstammdatenregister.de/MaStR>).

The power generation file contains data about each single power plant in Germany.

- Please calculate the accumulated power generation for each administrative district and plant type. (2P)
- Both datasets use the same IDs for the administrative districts. Merge the two datasets so that the entries are correctly matched by administrative districts for the plant type **wind**. (4P)
- Plot a map that encodes the generated wind power for each administrative district. Include a legend that tells the reader how to interpret the colors. (2P)
- Currently, the colormap is not very meaningful because there are two districts producing much more power than all other districts. Recreate the plot with optimized utilization of the colorscale range by clipping the upper bound. (2P)
- Employ **geopandas** to compute the size of each administrative district in Germany, and create a plot of the generated wind power, normalized by area. (3P)
- What pattern do you observe in the distribution of wind power generation across Germany? (1P)

Exercise 3 (Interactive Visualization with Dash, 25 Points)

Interaction is a key component of many visualizations. One tool for building interactive visualizations in the form of web applications is the Dash framework. The goal of this task, and a follow-up task on the next sheet, is to learn its basics. You should find the Dash tutorial, as well as the documentation chapter on Pattern Matching Callbacks, useful for solving this exercise (<https://dash.plotly.com/>).

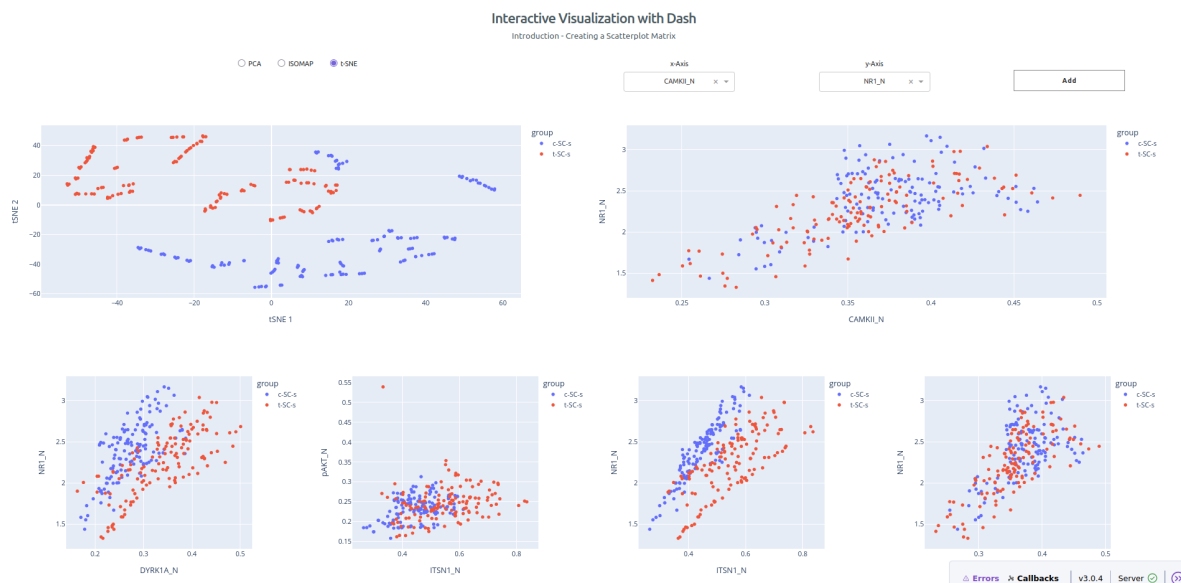


Figure 2: A screenshot of the interactive visualization you are asked to implement in this exercise.

- Read the file `Data_Cortex_Nuclear.xls` (from Sheet 3), and restrict it to the classes t-CS-s and c-CS-s. Run PCA, ISOMAP and t-SNE as dimensionality reduction techniques. Within the Dash framework, create a single scatter plot that will show the output from one of the techniques, as selected by the user. Add a **Radio** component from Dash to switch between the different techniques. A **callback** function is a function which is triggered by an event such as a mouse click, double click, or selection. In Dash, you can use `@app.callback` to define your event callback, with the dropdown **value** as the input. (10P)
- Add a second scatter plot with corresponding **dropdown** menus that should give you the opportunity to map any individual feature (i.e., protein) to any of the axes. (5P)
- Allow the user to show up to four scatterplots by implementing an “Add” button. Clicking on it should replicate the scatterplot that is on the right side of your previously implemented app, keeping the current user-defined axis settings. The new plots should appear below the old ones, as in Figure 2. To maintain a clean interface, adding a fifth plot should remove the oldest plot. (10P)

Hints:

- You can customize your app’s appearance using CSS and the `dash_bootstrap_components` module. If you use `plotly.express`, try:


```
external_stylesheets = [
    'https://codepen.io/chriddyp/pen/bWLwgP.css',
    dbc.themes.BOOTSTRAP,
    'https://use.fontawesome.com/releases/v5.12.1/css/all.css'
]
```

- If you haven't looked at States yet - now is the time to do so. See the Basic Callbacks Tutorial.
- All the components in a Div container are stored in its "children" list. (This assignment usually happens automatically when you define your layout.) You can achieve a flexible layout by giving the Div container a component ID and using its "children" list as both a State and an Output for your callback.
- If the callback was triggered by the Add button, you want to append the new plot to the children list of the parent Div.

Good Luck!